Kaushik G Kulkarni

Email: kgk2@illinois.edu
Web: https://kaushikcfd.github.io

Education

Aug '17 – July '23	Ph.D. in Computer Science University of Illinois at Urbana-Champaign, Urbana, IL *Domain-Specific Code Transformations for Computational Science based on the Polyhedral Model*
Aug '13 – May '17	Bachelor of Technology in Mechanical Engineering Indian Institute of Technology, Bombay

Experience

Aug '23 – *Present*	Senior Engineer *Qualcomm Technologies Inc., San Diego, CA.* Researching programming abstractions for high-throughput machine learning inference on Neural Processing Units.
Aug '20 – July '23	Research Assistant *National Center for Supercomputing Applications, UIUC*
Aug '17 – Aug' 20	Graduate Research/Teaching Assistant *Computer Science Dept., UIUC*
May '19 – Aug '19	Givens Associate *Argonne National Laboratory, Illinois* Improved the implementation of Nonlinear optimization algorithms on multi-core systems.
Jan '16 – May '17	Teaching Assistant for Introduction to Numerical Analysis *Mathematics Dept., IITB*
May '16 – July '16	Software Engineering Intern *Morgan Stanley Strats and Modelling, Mumbai* Developed internal tools to aid cash flow visualizations for traders.

Publications

[1] **K. Kulkarni** and A. Klöckner, *Canonicalization of batched einstein summations for tuning retrieval*, Submitted to ACM TOMS, 2026. arXiv: `2601.12220 [cs.MS]`. [Online]. Available: `https://arxiv.org/abs/2601.12220`.

[2] M. Diener, M. J. Smith, M. T. Campbell, **K. Kulkarni**, M. J. Anderson, A. Klöckner, W. Gropp, J. B. Freund, and L. N. Olson, *Mirge: An array-based computational framework for scientific computing*, Submitted to ACM TOMS, 2025. arXiv: `2512.17101 [cs.MS]`. [Online]. Available: `https://arxiv.org/abs/2512.17101`.

[3] **K. Kulkarni** and A. Klöckner, *Code generation for near-roofline finite element actions on gpus from symbolic variational forms*, Submitted to ACM TOMS, 2025. arXiv: `2506.17471 [cs.DC]`. [Online]. Available: `https://arxiv.org/abs/2506.17471`.

[4] T. Sun, L. Mitchell, **K. Kulkarni**, A. Klöckner, D. A. Ham, and P. H. Kelly, "A study of vectorization for matrix-free finite element methods," *The International Journal of High Performance Computing Applications*, vol. 34, no. 6, pp. 629–644, 2020. DOI: `10.1177/1094342020945005`.

[5] A. Bhati, R. Sawanni, **K. Kulkarni**, and R. Bhardwaj, "Role of skin friction drag during flow-induced reconfiguration of a flexible thin plate," *Journal of Fluids and Structures*, vol. 77, pp. 134–150, 2018.

Talks

[1] K. Kulkarni, "Transforming (not just) DG-FEM Array Expressions (not just) on GPUs," ser. Minisymposium on PDE Simulations with High-Productivity Languages at the Dawn of Exascale, SIAM Conference on Computational Science and Engineering, Amsterdam, Netherlands, 2023.

[2] K. Kulkarni, "Transforming (not just) DG-FEM Array Expressions (not just) on GPUs," UIUC Compiler Seminar, 2022.

[3] K. Kulkarni, "Transforming (not just) DG-FEM Array Expressions (not just) on GPUs," ser. Flexible, Performance Portable Software for Partial Differential Equations, SIAM Conference on Parallel Processing for Scientific Computing, online, 2022.

[4] K. Kulkarni, "UFL to GPU: Generating near-roofline action kernels," ser. Minisymposterium: Automatic Simulation Code Generation Tools and Their Applications, SIAM Conference on Computational Science and Engineering, online, 2021.

[5] K. Kulkarni, "UFL to GPU: Generating near-roofline actions kernels," FEniCS 2021, online, 22–26 March, 2021.

[6] K. Kulkarni, "Optimizing finite element operator evaluation on GPUs," Firedrake USA, Seattle, WA, 2020.

[7] K. Kulkarni, "Transformation-based code optimization for finite element methods," ser. Minisymposium on Performance Portability through Source-to-source code translation, SIAM Conference on Computational Science and Engineering, Spokane, WA, 2019.

Awards and Achievements

2023	SIAM Travel Award for Conference on Computational Science and Engineering (Amsterdam, NL)
2022	Graduate College Presentation Award to present at SIAM Conference on Parallel Processing for Scientific Computing
2021	SIAM Travel Award for Conference on Computational Science and Engineering
2019	SIAM Travel Award for Conference on Computational Science and Engineering (Spokane, WA)
2016	Undergraduate Research Award (URA01) by Indian Institute of Technology, Bombay
2013	*Kishore Vaigyanik Protsahan Yojana* Fellowship Award
2013	Certificate of Merit for being among the State Top 1% in National Standard examination in Physics and Chemistry

Research

Optimizing Einstein-Summation Subprograms Einstein-summations provide a simple notation to express a wide-range of Linear-Algebra primitives. Achieving roofline FLOP-throughput for these operations still remains challenging. We employ a combination of techniques from auto-tuning to pattern-matching to generate efficient OpenCL code for computational kernels containing expressions that have einsum-like memory access patterns.

Array Programming Languages and Intermediate Representations Array Programming Languages have been an important vehicle for driving scientific applications from as early as the 1960s. Besides providing a close-to-math expressibility, their intermediate representations are closer to SIMD architectures making it easier to engineer optimizing compilers targeting such hardwares. Pytato provides one such IR that lowers $n - d$ array programs to computation graphs comprising of *pure*-Array Ops that can be targeted to OpenCL / CUDA / JAX.

Near-Roofline Discontinuous Galerkin Action Operators Discontinuous Galerkin operator applications comprise of many fine-grained array operations that can push them into the memory-bound regime. With kernel and loop fusion we can bump up the workload's Arithmetic Intensity, however, performing fusion might also negatively affect the kernel's performance by inhibiting device's latency hiding abilities by further introducing dependency edges and increasing the working set size of the inner loops. In the MIRGE-Com framework we address such trade-offs for GPU systems.

Finite Element Assembly on GPUs Evaluation of Finite Element operators result in a diverse set of computational kernels making it a difficult problem to find one optimization strategy that achieves near-peak performance for all the kernels on GPUs. We solve this problem by using high level code generation tools that select the optimization strategy based on the loop structure of the kernel.

Key Software Contributions

P = Primary (Maintainer) | S = Significant contributions

Loopy^S	An intermediate representation and transformation toolkit for programs with single exit-point loops for targets with SPMD execution models. My work introduced the callables abstraction within the IR that eventually enabled it to interact with the computer algebra systems in projects like Firedrake, MIRGE-Com. Documentation: `https://documen.tician.de/loopy/`
Pytato^P	An intermediate representation for n-d array operations that supports the pure subset of Numpy operations. The IR can be targeted to Loopy IR, XLA/JAX, etc. Pytato has been used to deploy a production Discontinuous Galerkin solver which is capable of delivering roofline performance while a computational scientist writes Numpy-like code. Documentation: `https://documen.tician.de/pytato/`
feinsum^P	Toolkit for managing databases of code-transformations on Batched Einstein Summations. Unlike typical computational primitive libraries that ship optimized computational kernel in source or binary form, feinsum records code- transformations from a trivial schedule, resulting in relative savings from not having to flush the operands to the DRAM. Documentation: `https://feinsum.readthedocs.io/`
Arraycontext^P	Provides a subset of Numpy operations that are avaialable to both lazy and eager modes of execution. At runtime, these operations can be retargeted to array implementations, such as Pytato, JAX, PyOpenCL, etc. Documentation: `https://documen.tician.de/arraycontext/`.
symoxide^P	A computer algebra system written purely in Rust. Demonstrated $\sim$5 speedup w.r.t. state-of-the-art implementations in Python for expression traversals of production Scientific Computing workloads. Documentation: `https://docs.rs/symoxide`
ISL-Rs^P	Rust bindings for Integer Set Library to access polyhedral analysis primitives, where the implementation is responsible for automatic cleanup of objects as per Rust semantics. Documentation: `https://docs.rs/isl-rs`.
pybliss^P	Idiomatic Python bindings for the graph isomorphism toolkit, Bliss. Documentation: `https://pybliss.readthedocs.io`.
Miscellaneous	Minor improvements to PETSc, PyCUDA, Firedrake, etc.

Last updated on April 8, 2026